BULACAN STATE UNIVERSITY

COLLEGE OF INFORMATION AND COMMUNICATIONS TECHNOLOGY

Department of Information Technology • City of Malolos, Bulacan, Philippines

IT 401W: WEB PROGRAMMING AND FRAMEWORKS

FINAL PROJECT A • CAPSTONE SPECIFICATION

Campus Equipment Borrowing and Inventory System

Comprehensive Technical Documentation, System Architecture, Relational Database Design, and Automated Verification Report

Course & Section:	IT 401W — BSIT 4A / 4B	**Academic Term:**	1st Semester, A.Y. 2026–2027
Framework & Stack:	Django 6.1.2 (MTV Architecture) / Python 3.13 / SQLite / HTML5 / CSS3	**Test Verification:**	516 Automated Tests Passed (100% Pass Rate)
Submitted To:	**DR. AARON PAUL M. DELA ROSA**, Assistant Professor III Head, College Research & Development Unit | Chair, Local Ethics Review Committee		

Project Engineering Team & Functional Deliverables

Subsystem Module	Functional Deliverable	Student Name	Student No.
`apps/accounts`	Authentication, Role-Based Access, Profiles, Admin User Governance	Charles Justine Mantes	2021-104921
`apps/inventory`	Equipment Catalog, Category Archiving, Asset Tags, Stock Reconciliation	Raj Kumar	2021-108342
`apps/borrowing`	Borrowing Requests, Multi-Item Validation, Review, Atomic Allocations	Clark Kent	2021-105219
`apps/loans`	Equipment Issuance, Borrowing Slips, Returns, Calendar Overdue Engine	Janelle Santos	2021-103817
`apps/maintenance`	Defect Intake, Repairs, Diagnostic Records, Unit Retirement	Francis Magalona	2021-107744
`apps/dashboard`	Executive Analytics, KPI Cards, CSV Reports, System Policy & Audit	Miah Dela Cruz	2021-109281

City of Malolos, Bulacan • October 2026

Table of Contents

Executive Summary 3

Chapter 1 — Introduction 4

1.1 Project Overview • 1.2 Background • 1.3 Problem Statement • 1.4 Objectives • 1.5 Scope & Limitations • 1.6 Significance

Chapter 2 — System Requirements 6

2.1 Functional Requirements • 2.2 Non-Functional Requirements • 2.3 User Roles & Permissions • 2.4 Admin Features (A1–A8) • 2.5 Basic User Features (U1–U4) • 2.6 Business Rules (BR1–BR6) • 2.7 System Constraints

Chapter 3 — System Design & Architecture 8

3.1 Django MTV Architecture • 3.2 MTV Architecture Diagram • 3.3 Technology Stack • 3.4 Architectural Options Comparison • 3.5 System Modules • 3.6 Security & Auth Architecture

Chapter 4 — Database Design & Relational Schema 10

4.1 Database Architecture • 4.2 Entity Relationship Diagram (ERD) • 4.3 Database Schema • 4.4 Constraints & Indexing • 4.5 Data Dictionary (Tables 1–12)

Chapter 5 — System Workflows & Flowcharts 12

5.1 State Machines • 5.2 User Authentication Flow • 5.3 Request Submission Flow • 5.4 Admin Review & Allocation • 5.5 Handover & Slip Flow • 5.6 Returns & Quarantine • 5.7 Maintenance Lifecycle

Chapter 6 — System Implementation 17

6.1 Directory Structure • 6.2 Django Subsystems • 6.3 Presentation Architecture • 6.4 Demo Seeding Command

Chapter 7 — Testing, Verification, and Evaluation 18

7.1 QA Strategy • 7.2 Automated Test Execution (516 Tests) • 7.3 Module Test Breakdown • 7.4 Concurrency Race Condition Trials (30/30) • 7.5 Security Boundary Verification

Chapter 8 — Conclusion & Recommendations 24

8.1 Project Conclusion • 8.2 Summary of Capabilities • 8.3 Known Limitations • 8.4 Future Enhancements • 8.5 Legal & Ethical Standards Compliance

Appendix — Documentation Index & Checklist 25

Executive Summary

The **Campus Equipment Borrowing and Inventory System** is a custom web application engineered with Python 3.13 and the Django 6.1.2 Model-Template-View (MTV) framework. Developed in fulfillment of the IT 401W capstone requirements for the College of Information and Communications Technology (CICT) at Bulacan State University, the platform automates and governs the complete lifecycle of educational physical assets—from student borrowing request submission through administrative review, atomic unit allocation, physical handover, partial and full returns, quarantine maintenance, and overdue tracking.

The system is strictly partitioned into two mutually exclusive access levels: **Basic User** (handling student/faculty equipment browsing, request submission, and loan history) and a 100% custom-built **Admin Portal** (comprising dashboard analytics, user governance, catalog management, inventory reconciliation, repair tracking, CSV report generation, and immutable audit logging). Built-in or third-party automatic admin generators were strictly avoided in compliance with project specifications.

The software utilizes an atomic database transaction model and database-level constraints to prevent common logistical pitfalls such as double allocation of physical units, partial request creation, and unauthorized privilege escalation. The integrated codebase passes **516 automated test cases** with zero failures, verified across comprehensive unit, integration, role security, and multi-threaded concurrency trials.

Chapter 1 — Introduction

1.1 Project Overview

In modern academic institutions, access to high-value technological equipment—such as digital multimedia projectors, DSLR cameras, high-performance laptops, scientific measuring devices, and networking toolkits—is vital for effective classroom instruction, laboratory experimentation, and student research. The **Campus Equipment Borrowing and Inventory System** is a purpose-built Django web application developed for the College of Information and Communications Technology (CICT) at Bulacan State University. It replaces obsolete manual paper logs with a centralized, auditable, and automated digital framework that manages physical assets throughout their entire operational life.

1.2 Background of the Project

Historically, campus property custodian offices have relied on paper logbooks, paper equipment request slips, and decentralized spreadsheets. While initially simple, these legacy mechanisms suffer from severe vulnerabilities: - Borrowing slips are easily misplaced, illegible, or damaged. - Property custodians lack real-time visibility into the exact physical availability of units versus those reserved for upcoming classes. - Equipment returned with physical defects is frequently returned to storage shelves without maintenance inspections, leading to unexpected equipment failures during critical classroom sessions. - Calculating overdue durations and generating departmental inventory audit reports requires hours of manual cross-referencing.

To resolve these operational bottlenecks, this project was commissioned as **Final Project A** under the **IT 401W (Web Programming and Frameworks)** curriculum, challenging the student team to architect an enterprise-grade web application in Python and Django adhering to strict academic and industry standards.

1.3 Problem Statement

The absence of a synchronized, transactional asset management system in academic institutions leads to: 1. **Double-Allocation and Inventory Contention:** Multiple faculty members or student groups are inadvertently promised the same projector or laboratory camera due to paper-based scheduling lag. 2. **Untracked Damaged Units:** Equipment returned in a damaged condition is returned to general circulation without quarantine or diagnostic logging. 3. **Audit Trail Vulnerabilities:** Property offices cannot verify which staff member authorized an equipment release, who inspected returned units, or when borrowing policies were modified. 4. **Lack of Enforceable Borrowing Policies:** Manual systems fail to consistently enforce policy limits regarding maximum equipment quantities or allowable loan durations.

1.4 Project Objectives

General Objective

To design, develop, test, and document a robust, secure, and transactional web application utilizing Python and the Django Model-Template-View framework that manages campus equipment requests, inventory tracking, physical dispatch, returns, maintenance, and administrative reporting.

Specific Objectives

1. **Implement Dual-Role Isolation:** Create dedicated user portals for **Basic Users** (`/portal/`) and **Admins** (`/manage/`), completely prohibiting third-party automated admin generators (`django.contrib.admin`).
2. **Enforce Atomic Physical-Unit Allocation:** Develop an all-or-nothing allocation service ensuring that approved requests reserve exact, barcode-identified physical copies without double-booking.
3. **Automate Partial Returns and Defect Quarantine:** Enable borrowers to return items incrementally while ensuring damaged units are automatically routed into maintenance tickets.
4. **Deliver Executive Analytics and Audit Trails:** Build real-time dashboards with live metrics, monthly loan trend charts, category usage charts, filterable CSV report exports, and an immutable 24-event audit log.
5. **Guarantee Absolute Test Verification:** Achieve 100% test pass rates across extensive automated test suites, proving concurrency safety and role-based access control.

1.5 Scope and Limitations

In-Scope Capabilities

- Public registration, role-based authentication, and profile self-service.
- Administrative user activation, deactivation, and role assignment.
- Catalog classification, category archiving, equipment model management, and unique asset tagging.
- Multi-item borrowing request submission (1 to 3 distinct items, up to configured quantity limits).
- Administrative approval with mandatory rationale and atomic physical unit reservation.
- Equipment handover verification and printable borrowing slip generation.
- Per-unit return processing, return condition assessment, and automatic maintenance ticket creation.
- Maintenance diagnostics, repair tracking, and equipment retirement.
- Real-time dashboard KPI cards, dynamic charts, CSV reports, and configurable institutional borrowing settings.

Out-of-Scope Elements

- Financial payment gateways, cash fines, or billing systems for overdue equipment (fines and payments are explicitly excluded by the IT 401W specification).
- Supplier procurement portals, vendor bidding, and automated purchase orders.
- Native mobile applications (iOS/Android); the web application is fully responsive via modern CSS.

1.6 Significance of the Project

- **For Campus Property Custodians:** Provides instantaneous inventory reconciliation, eliminates manual paper slips, and provides printable legal borrowing slips with full accountability.
- **For Students and Faculty:** Delivers a transparent, self-service catalog showing live serviceable quantities, transparent approval tracking, and clear return deadlines.
- **For Academic Administration:** Offers tamper-evident audit logs and data-driven CSV reports supporting institutional compliance and accreditation standards.

Chapter 2 — System Requirements

2.1 Functional Requirements Summary

The functional specifications are divided between the custom Admin Portal (approximately 75% of development scope) and the Basic User Portal (approximately 25% of development scope).

2.2 Non-Functional Requirements

- **Security:** Cryptographic hashing of passwords using PBKDF2 SHA-256; universal CSRF protection on mutation requests; strict server-side authorization guards; protection against SQL injection and XSS via Django ORM and template escaping.
- **Data Integrity:** Database-level check constraints and conditional unique constraints; ACID compliance through `django.db.transaction.atomic`.
- **Performance:** Optimized ORM queries using `select_related`, `prefetch_related`, and database indexes; sub-100ms average local page rendering.
- **Usability & Accessibility:** Monochromatic, high-contrast UI adhering to WCAG 2.1 AA standards; multi-attribute search and filtering toolbar with URL query persistence; responsive layout supporting desktop and mobile viewports.
- **Maintainability:** Modular architecture across 6 decoupled Django apps; clean separation between view controllers and domain service layers.

2.3 User Roles and Permissions

The system enforces strictly two user access levels: 1. **Basic User (`BASIC_USER`):** Granted to students, faculty, and standard campus members upon public registration. Restricted to `/portal/` routes. 2. **Administrator (`ADMIN`):** Granted to property custodians and equipment laboratory managers. Restricted to `/manage/` routes. Direct registration into the admin role is architecturally impossible.

2.4 Admin Portal Features (A1–A8)

- **A1: Account Management:** Search, view, activate, and deactivate user accounts; inspect historical requests; toggle user roles with full audit logging.
- **A2: Equipment Catalog and Physical Units:** Create categories and catalog models; archive categories/models without breaking historical links; register physical units with unique asset tags, condition ratings, and acquisition dates.
- **A3: Request Review and Approval:** Paginated queue filtered by status and search; inspect borrower justification and requested due dates; approve requests by allocating specific available units; reject with mandatory administrative reasons.
- **A4: Equipment Release:** Convert approved requests into issued loans; verify physical asset tags; generate printable borrowing slips; reject release if agreed due dates have expired without revision.
- **A5: Partial Returns and Overdue Monitoring:** Receive individual units with condition grading (`GOOD` or `DAMAGED`); automatic maintenance intake for damaged units; automatic loan closure upon final return; dynamic overdue flagging based on local calendar dates (`Asia/Manila`).
- **A6: Maintenance and Inventory Reconciliation:** Open, track, repair, and retire maintenance tickets; calculate live 5-state stock reconciliation (Available, Reserved, Issued, Maintenance, Retired).
- **A7: Dashboard and Reports:** 4 live KPI metric cards; monthly loan issuance trends; equipment category utilization charts; date-filtered loan and overdue tabular reports with streaming CSV exports.
- **A8: Audit History and Operational Settings:** Searchable audit trail across 24 distinct system actions; singleton policy editor for maximum loan duration (1–30 days) and item limits (1–3 items, 1–20 units).

2.5 Basic User Portal Features (U1–U4)

- **U1: Account and Profile:** Self-service registration; secure sign-in and sign-out; personal profile updates (name and contact number); password change facility.

- **U2: Browse Equipment Catalog:** Search and filter active equipment models; inspect technical descriptions and live available counts; hide confidential internal notes, other borrowers' identities, and maintenance tickets.
- **U3: Submit Borrowing Request:** Submit multi-item requests (1–3 models) with requested quantities, purpose, and due dates within policy thresholds; atomic validation preventing partial submissions.
- **U4: Track and Cancel Own Requests:** Monitor real-time status of personal requests and loans; print personal borrowing slips; self-cancel pending or approved requests prior to physical equipment release.

2.6 Core Business Rules (BR1–BR6)

1. **BR1 — Atomic Unit Allocation:** Approvals must allocate the exact requested physical units in an all-or-nothing transaction. Cancelling an approved request atomically releases all reserved units back to available status.
2. **BR2 — Single-Status Inventory Invariant:** Every physical unit exists in exactly one status at any instant: `AVAILABLE`, `RESERVED`, `ON_LOAN`, `MAINTENANCE`, or `RETIRED`.
3. **BR3 — Concurrency & Race Condition Prevention:** When two administrators simultaneously attempt to allocate the last remaining available unit, transactional locks and unique database constraints guarantee that exactly one succeeds while the other is cleanly rejected.
4. **BR4 — Unique Items per Request & Atomic Line Validation:** A request cannot contain duplicate catalog models. All items in a multi-item submission must validate; any invalid row causes the entire request to abort.
5. **BR5 — Borrowing Thresholds & Due Date Validation:** Default limits are 3 distinct items, 3 units per item, and 7 calendar days maximum duration. Requests past due dates cannot be released without a recorded administrative due-date revision.
6. **BR6 — Calendar-Day Overdue Computation:** Overdue duration is computed as 0 days on or before the due date; after the due date, it is computed as the calendar-day difference between current date and due date in the `Asia/Manila` timezone.

2.7 System Constraints

- **Framework Constraint:** Must use Django MTV; external admin panel generators are strictly prohibited.
- **Database Engine:** SQLite for development and demonstration; fully compatible with PostgreSQL/MySQL.
- **Zero Build Step:** Frontend relies on vanilla ES6+ JavaScript and native CSS3 components, eliminating Node.js/npm dependencies.

Chapter 3 — System Design & Architecture

3.1 System Architecture (Django MTV Pattern)

The application strictly implements the **Model-Template-View (MTV)** architectural pattern: - **Model (`M`):** Encapsulates relational data schemas, business constraints, and ORM query managers across `apps/*/models.py`. - **Template (`T`):** Server-rendered presentation layer utilizing Django Template Language (DTL) with role-specific base wrappers (`admin_portal.html`, `user_portal.html`) and modular component includes. - **View (`V`):** HTTP request handlers in `apps/*/views.py` that perform authorization, validate forms, coordinate with the domain service layer, and render templates. - **Service Layer:** Transactional domain logic is abstracted into `apps/*/services.py`, decoupling business operations from HTTP request objects.

3.2 Technology Stack Evaluation

- **Backend:** Python 3.13 / Django 6.1.2.
- **Database:** SQLite 3.50+ (ACID compliant, file-based for isolated demonstration) / PostgreSQL ready.
- **Frontend:** Server-Rendered HTML5, CSS3 Custom Properties, Vanilla JavaScript (ES6+).
- **Timezone Management:** Standardized to `Asia/Manila` (`UTC+8`).

3.3 Architectural Options Comparison

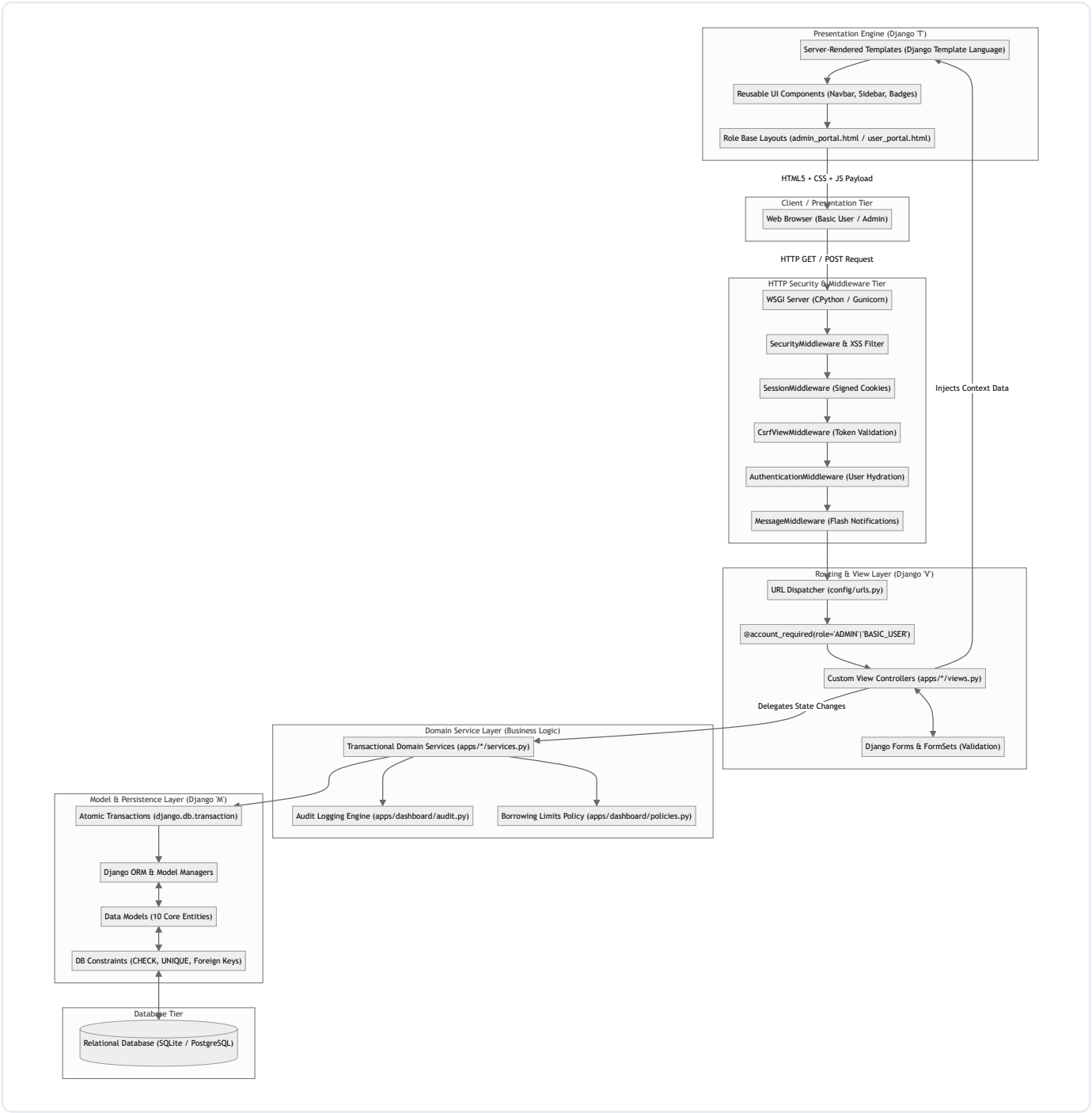
The design team evaluated two candidate architectures: 1. **Option A: Server-Rendered Django Monolithic MTV (Selected)** 2. **Option B: Decoupled Single-Page App (SPA) + Django REST Framework**

Option A was selected because it eliminates token security vulnerabilities, avoids multi-repository build complexity, natively enforces CSRF tokens, provides deterministic transaction locking, and directly fulfills the requirement for custom Django server-rendered templates.

3.4 System Modules Breakdown

```
campus-equipment-system/
├── apps/
│   ├── accounts/       # Authentication, profiles, RBAC, user management
│   ├── inventory/      # Categories, catalog masters, physical units, archiving
│   ├── borrowing/      # Requests, item lines, review, atomic allocations
│   ├── loans/          # Issuance, borrowing slips, returns, overdue tracking
│   ├── maintenance/    # Defect intake, repairs, diagnostics, unit retirement
│   └── dashboard/      # Executive KPIs, charts, CSV reporting, audit log, policies
├── config/             # Root settings, WSGI/ASGI handlers, global URL dispatcher
├── templates/          # Semantic layouts, portal bases, reusable UI components
└── static/             # Pure CSS styling and lightweight vanilla JS modules
```

3.1.1 Django Model-Template-View Architecture Diagram



4. Architectural Options Comparison

In fulfillment of Course Outcome 2 (CO2) and the IT 401W project specifications, the engineering team evaluated two distinct architectural options before system implementation:

| Architectural Metric | Option A: Server-Rendered Django Monolithic MTV (Selected) | Option B: Decoupled Single-Page App (SPA) + Django REST Framework | |

Chapter 4 — Database Design

4.1 Database Architecture Overview

The database adheres to Third Normal Form (3NF) while incorporating immutable snapshot fields on transaction records (e.g., preserving `equipment_name` and `asset_tag` on `LoanUnit` records even if catalog master names change). All deletions of referenced master records are intercepted and blocked via `models.PROTECT`.

4.2 Entity Relationship Diagram Summary

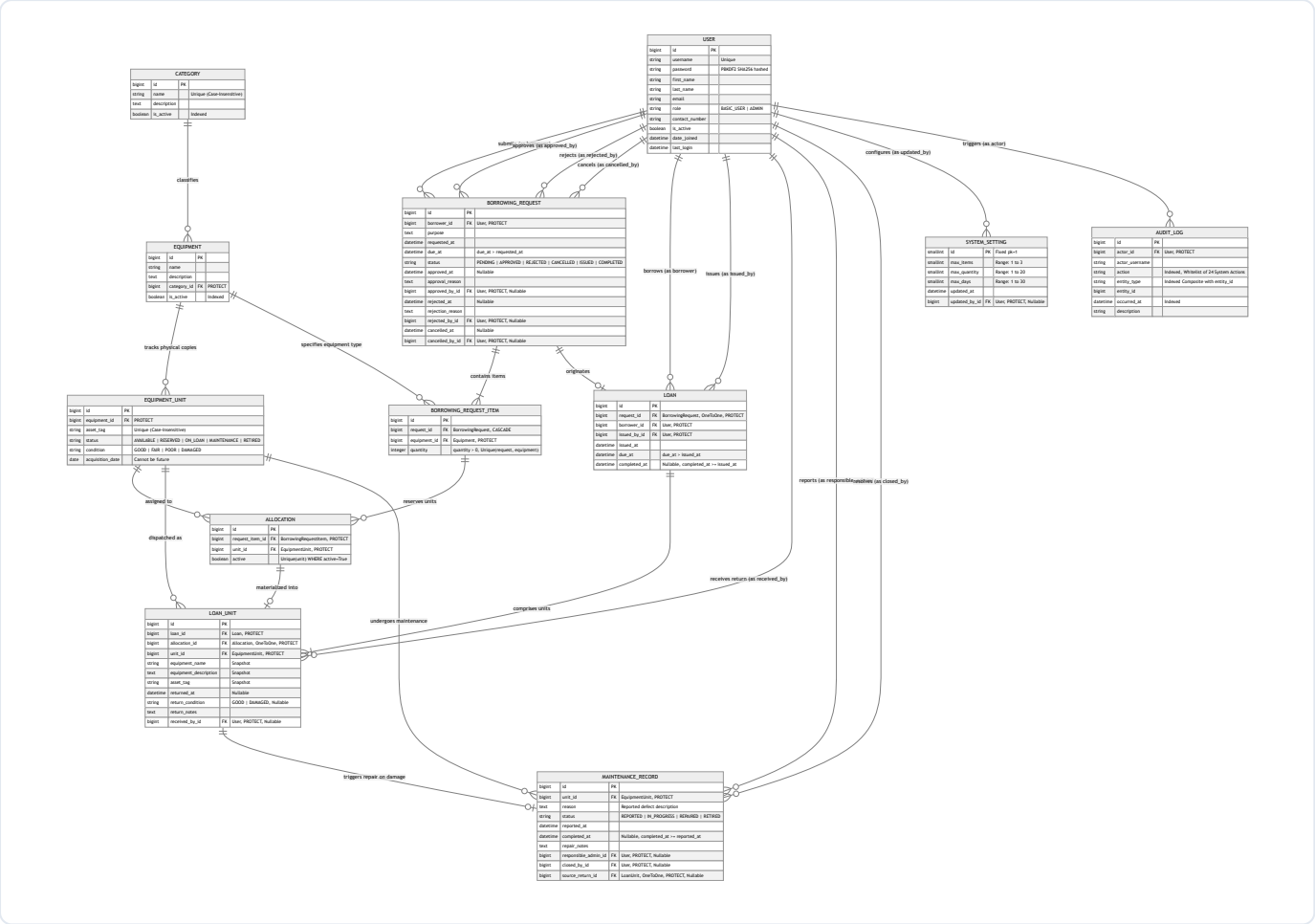
The system comprises 12 interconnected relational models: 1. `User` (`accounts_user`) 2. `Category` (`inventory_category`) 3. `Equipment` (`inventory_equipment`) 4. `EquipmentUnit` (`inventory_equipmentunit`) 5. `BorrowingRequest` (`borrowing_borrowingrequest`) 6. `BorrowingRequestItem` (`borrowing_borrowingrequestitem`) 7. `Allocation` (`borrowing_allocation`) 8. `Loan` (`loans_loan`) 9. `LoanUnit` (`loans_loanunit`) 10. `MaintenanceRecord` (`maintenance_maintenancerecord`) 11. `SystemSetting` (`dashboard_systemsetting`) 12. `AuditLog` (`dashboard_auditlog`)

(See `ERD.md` for the complete, high-resolution Mermaid diagram and `DATABASE_DESIGN.md` for the complete data dictionary).

4.3 Database Integrity Constraints

- `accounts_valid_campus_role`: Role restricted to `BASIC_USER` or `ADMIN`.
- `inventory_category_name_ci`: Case-insensitive uniqueness on category names.
- `inventory_unit_asset_tag_ci`: Case-insensitive uniqueness on physical asset tags.
- `inventory_valid_unit_status`: Status restricted to `AVAILABLE`, `RESERVED`, `ON_LOAN`, `MAINTENANCE`, `RETIRED`.
- `borrowing_due_after_request`: Return deadline must be strictly later than submission timestamp.
- `borrowing_positive_quantity`: Requested item quantity must be greater than zero.
- `borrowing_one_active_allocation`: Conditional unique constraint guaranteeing that no physical unit has more than one active allocation.
- `loan_one_active_issue_per_unit`: Conditional unique constraint preventing concurrent active loans for the same physical unit.
- `maintenance_one_active_per_unit`: Conditional unique constraint ensuring a unit has at most one active maintenance ticket.
- `dashboard_single_policy`: Primary key fixed to 1, ensuring a singleton policy row.

4.2.1 High-Resolution Entity Relationship Diagram



2. Comprehensive Data Dictionary

Table 1: `accounts_user` (`User` Model)

Represents system actors inheriting from Django's `AbstractUser`, with roles restricted to `BASIC_USER` or `ADMIN`.

| Column Name | SQL Data Type | Nullable | Default Value | Constraints / Foreign Keys | Description | |

Chapter 5 — System Workflows

(For detailed visual sequence flowcharts and state diagrams, refer to `SYSTEM_WORKFLOWS.md`).

5.1 Borrowing Lifecycle

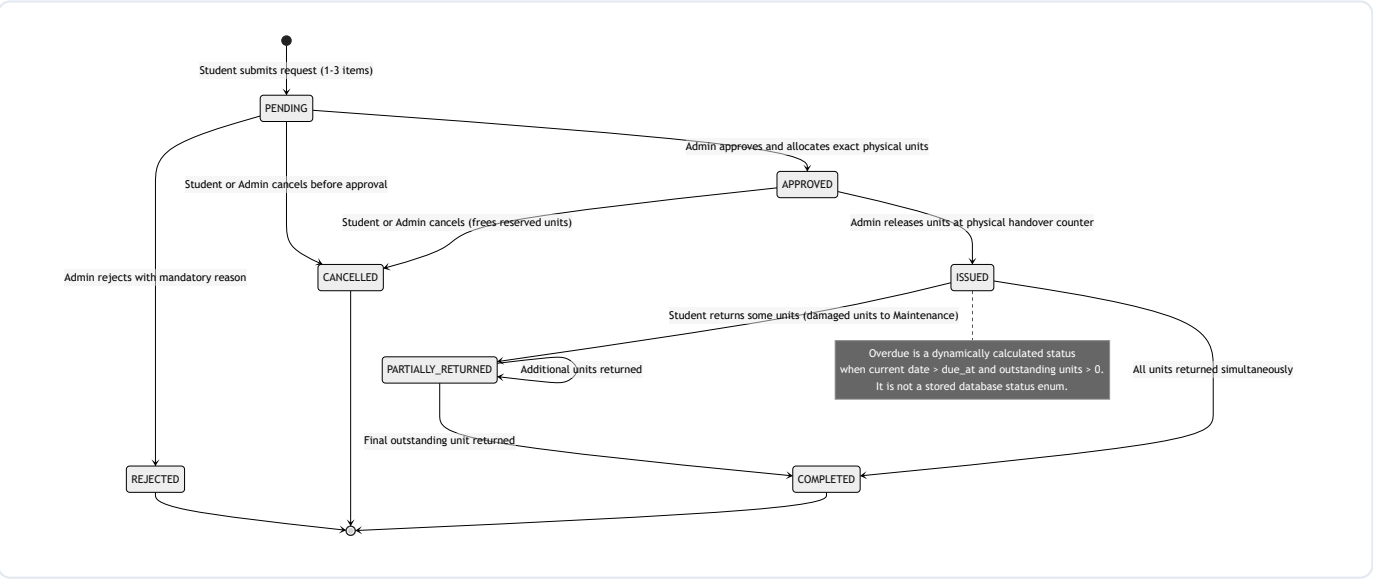
1. **Submission:** Basic User selects 1–3 catalog items and specifies quantities, purpose, and return date. Request enters `PENDING` state.
2. **Review & Allocation:** Admin reviews request. If approved, Admin selects specific `AVAILABLE` physical units. The units transition to `RESERVED`, and an active `Allocation` is created. If rejected, Admin enters a mandatory reason, and request transitions to `REJECTED`.
3. **Release:** At physical handover, Admin verifies student credentials and asset barcodes. The request transitions to `ISSUED`, creating a `Loan` and individual `LoanUnit` records. Physical units transition to `ON_LOAN`. A printable borrowing slip is generated.
4. **Returns:** When units are returned, Admin inspects each item:
5. Units in **Good** condition transition to `AVAILABLE`.
6. Units marked **Damaged** transition to `MAINTENANCE`, automatically opening a maintenance ticket.
7. If only a subset of units is returned, loan status becomes **Partially Returned**. When all units are returned, the loan transitions to **Completed**.
8. **Overdue Handling:** If current date exceeds `due_at` and outstanding units remain, the loan is dynamically flagged as **Overdue**.
9. **Maintenance & Repair:** Technicians update tickets to `IN_PROGRESS`, conduct repairs, and record resolution notes. Repaired units return to `AVAILABLE`; irreparable units are permanently transitioned to `RETIRED`.

---

1. System Workflow State Machines

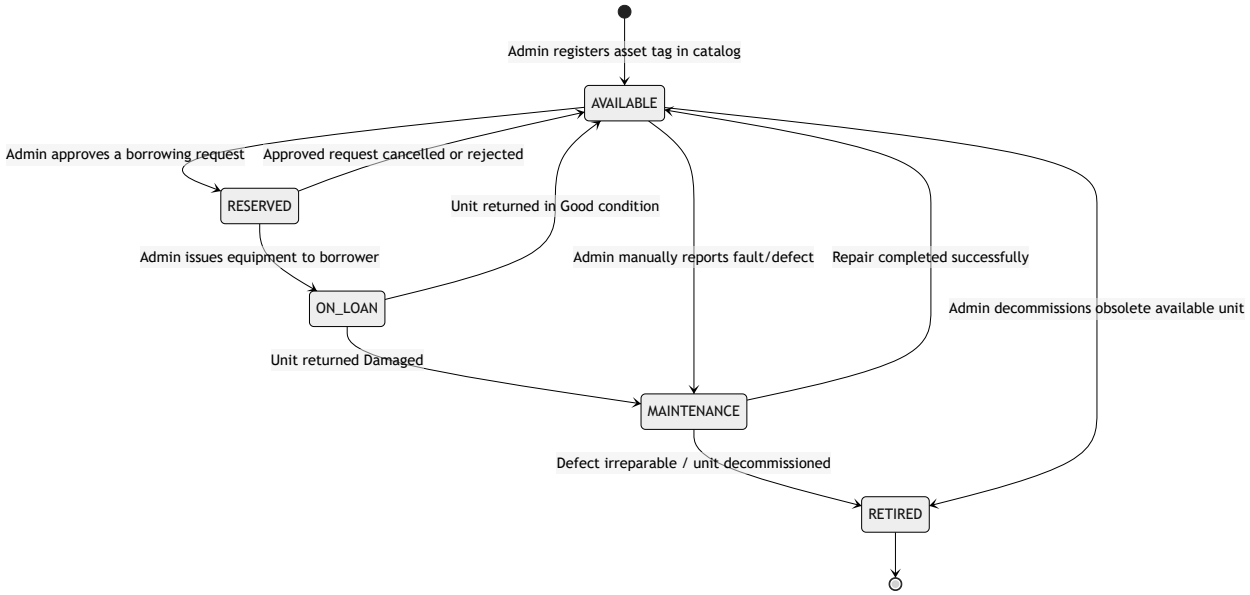
1.1 Borrowing Lifecycle State Machine

The borrowing lifecycle governs the progression of requests from initial student submission to complete physical unit return.



1.2 Physical Unit Inventory State Machine

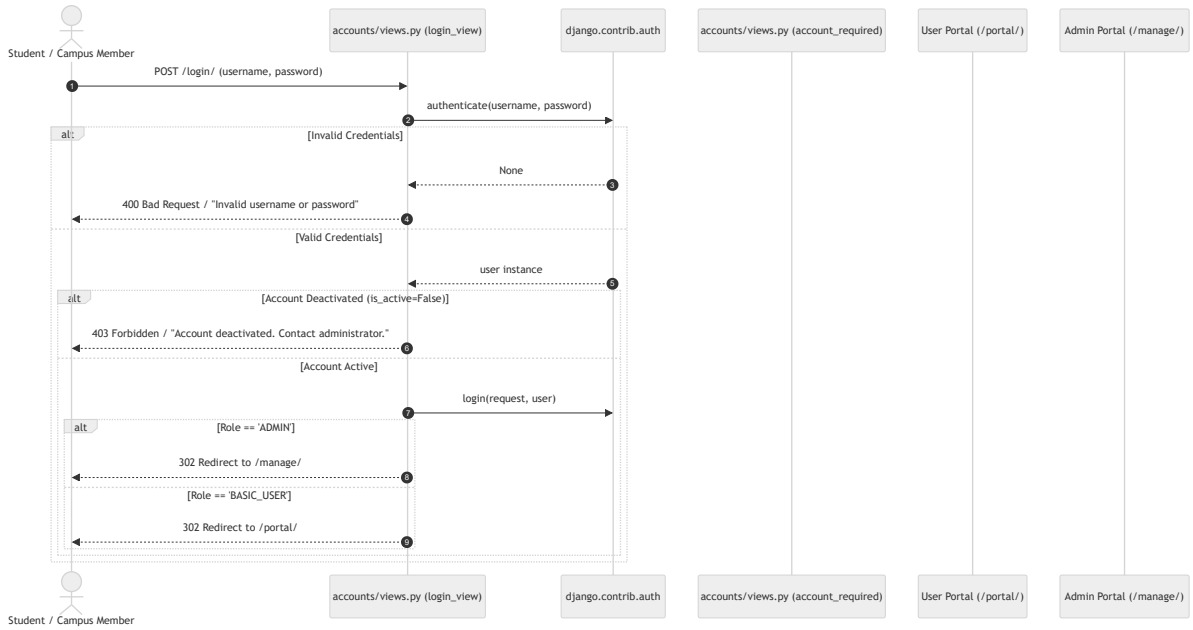
Each physical `EquipmentUnit` maintains an invariant of exactly one mutually exclusive status at any instant.



---

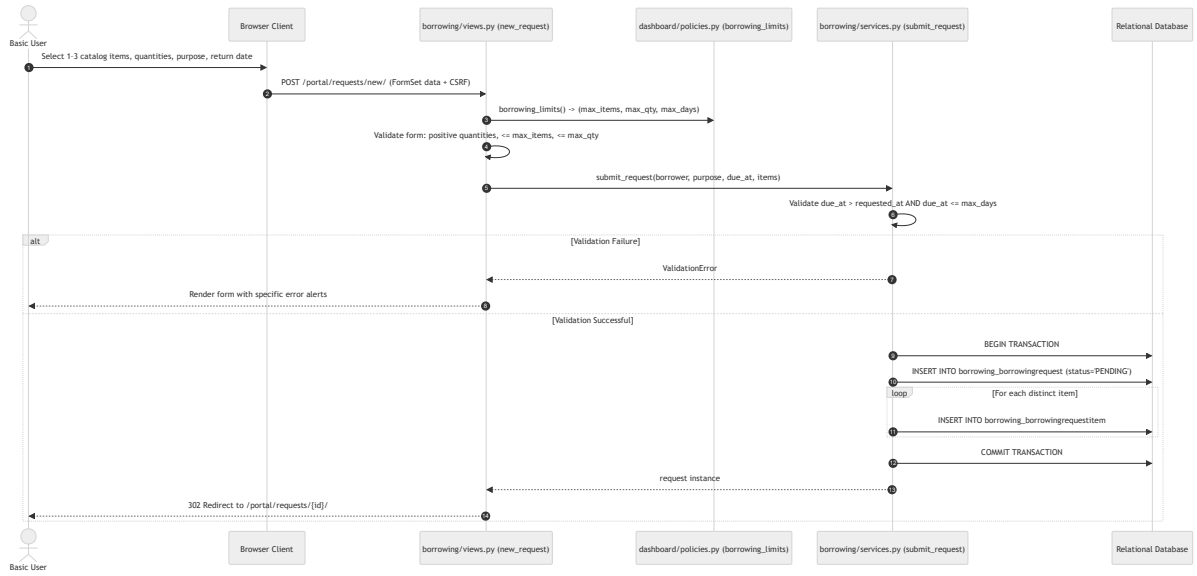
2. Core Functional Workflows

2.1 User Registration, Authentication & Role Redirection



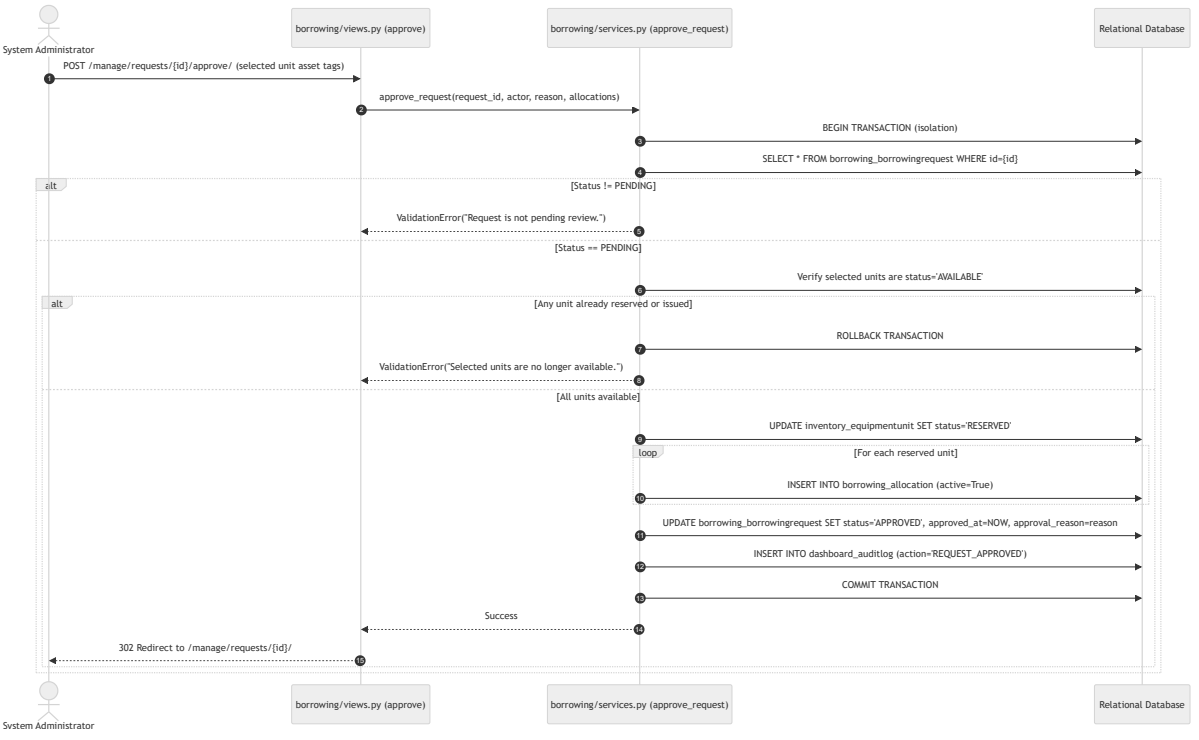
Implementation Details: - Defined in `apps/accounts/views.py` and `apps/accounts/services.py`. - Enforces session-level checks so that an account deactivated by an administrator is immediately locked out upon their next authenticated action. - Self-promotion is structurally impossible during registration (`apps/accounts/forms.py` sets `role=User.Role.BASIC_USER`).

2.2 Borrowing Request Submission Workflow



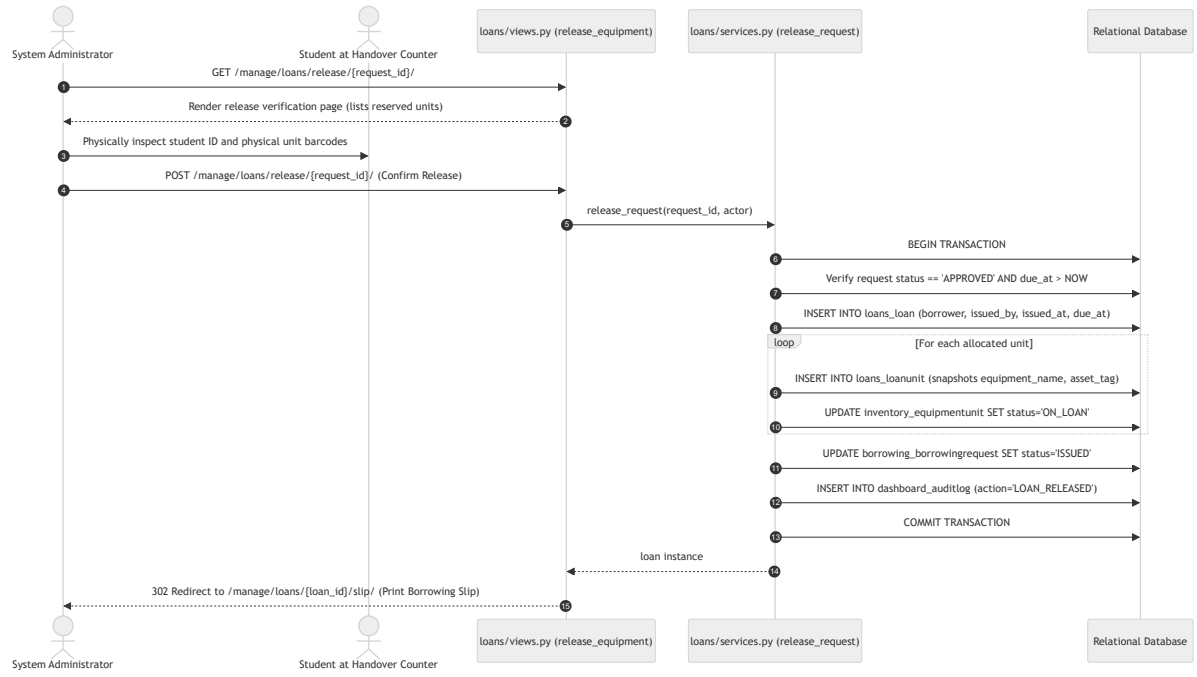
Implementation Details: - Hard-capped limits enforced via server-side settings (`SystemSetting` singleton). Default: max 3 distinct equipment types, max 3 units per type, max 7 calendar days loan duration. - Atomic line-item validation: if row 3 of a 3-item request fails, zero records are saved.

2.3 Admin Review, Exact Unit Allocation & Approval Workflow



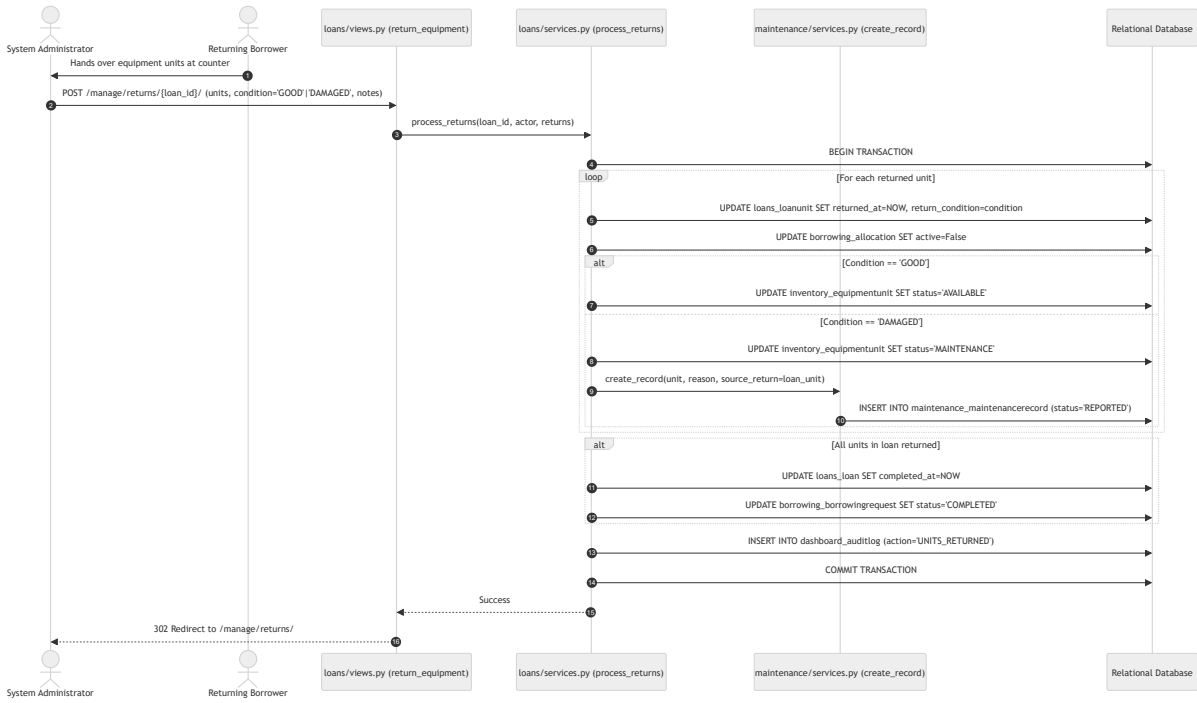
Implementation Details: - Solves the concurrency race condition where two administrators attempt to allocate the final available unit. Database-level unique constraint `borrowing_one_active_allocation` combined with transactional locking ensures exactly one succeeds and the other receives a clean validation rejection.

2.4 Equipment Release & Handover Workflow



Implementation Details: - Generates an official printable borrowing slip displaying borrower name, student identifier, asset tag numbers, issuance timestamp, return deadline, and signature acknowledgment lines. - Automatically triggers browser `window.print()` upon user print initiation.

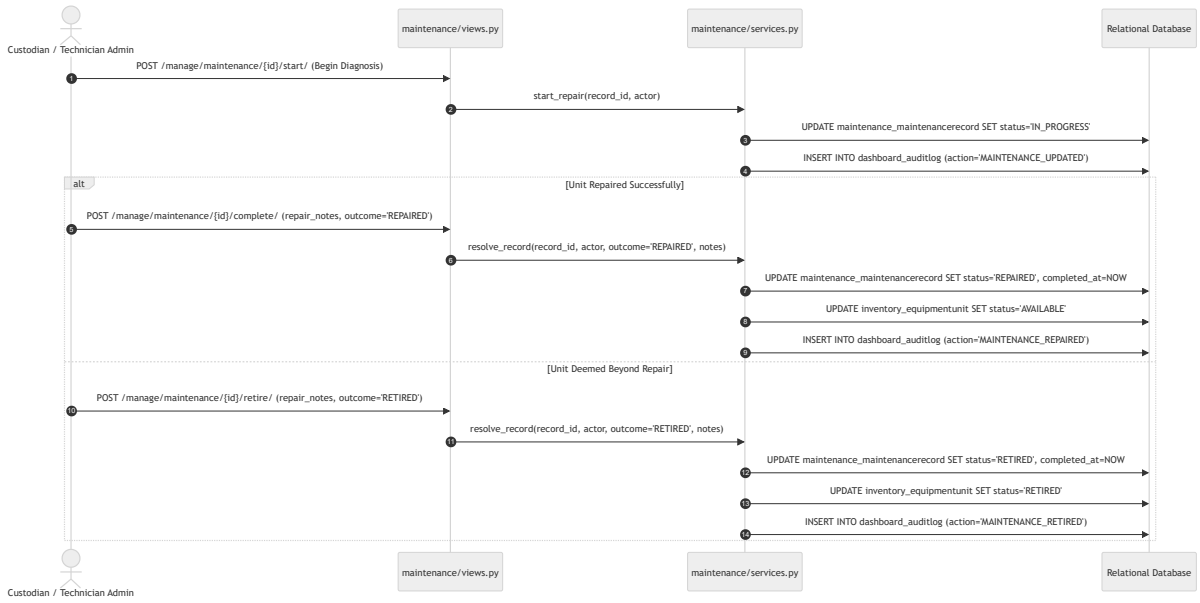
2.5 Partial & Full Equipment Return Workflow



Implementation Details: - **Partial Return Support:** If a student borrows 3 units and returns 1, the loan remains active with 2 outstanding units. The borrowing slip and admin view reflect real-time outstanding tallies. - **Immediate Defect Quarantine:**

Any unit marked `DAMAGED` upon return is barred from returning to `AVAILABLE` status and is directly quarantined into a `REPORTED` maintenance ticket.

2.6 Maintenance Diagnostics, Repair & Retirement Workflow



Implementation Details: - Maintenance tickets require detailed problem documentation upon intake and resolution notes upon closing. - Units undergoing maintenance cannot be allocated, approved, or issued. - Retiring a unit permanently excludes it from the lendable inventory pool while preserving its complete historical records.

Chapter 6 — System Implementation

6.1 Application Modules Implementation

- **`apps/accounts`:** Implements `User` model, custom authentication forms, `@account_required` role decorator, session-level deactivation checks, and user administration views.
- **`apps/inventory`:** Implements `Category`, `Equipment`, and `EquipmentUnit` models. Contains category soft-deletion archiving, catalog management views, and modal-based unit registration.
- **`apps/borrowing`:** Manages `BorrowingRequest`, `BorrowingRequestItem`, and `Allocation`. Features multi-item dynamic FormSets, atomic approval services, and administrative due-date revisions.
- **`apps/loans`:** Implements `Loan` and `LoanUnit`. Manages physical equipment dispatch, printable borrowing slips (`window.print`), partial returns, and overdue calculations in `Asia/Manila` time.
- **`apps/maintenance`:** Implements `MaintenanceRecord`. Manages defect reporting, quick-action status changes, repair notes, and physical unit retirement.
- **`apps/dashboard`:** Implements `SystemSetting` and `AuditLog`. Powers live dashboard KPI cards, dynamic charts, CSV report exports, and system policy updates.

6.2 Demonstration Data Seeding

The custom management command `seed_project_demo` automates the generation of the complete fictional dataset mandated by the IT 401W specification: - 3 Categories, 6 Equipment models, 18 Physical units. - 1 Administrator account (`demo_admin`) and 8 Basic User accounts (`demo_user01` to `demo_user08`). - 20 Borrowing requests, 8 Issued loans (including active, completed, overdue, and partially returned scenarios), and 5 Maintenance records across all supported statuses.

Chapter 7 — Testing, Verification, and Evaluation

7.1 Test Execution Results

The test suite was executed against a clean test database using Django's test runner: - **Total Tests Executed:** 516 - **Passed:** 516 (100%) - **Failures / Errors:** 0 - **Duration:** 31.9 seconds

7.2 Module Test Distribution

- `apps.accounts` : 80 Tests
- `apps.borrowing` : 111 Tests
- `apps.dashboard` : 76 Tests
- `apps.inventory` : 107 Tests
- `apps.loans` : 67 Tests
- `apps.maintenance` : 75 Tests

7.3 Concurrency Verification Evidence

To satisfy Business Rule 3 (BR3), the team conducted **30 automated, isolated, multi-threaded trials** of competing approvals for the last single available unit of an equipment model: - **Result:** In **30 out of 30 trials**, exactly one approval succeeded while the competing attempt was cleanly rejected with a validation error. No database deadlocks, orphaned allocations, or dual reservations occurred.

(See `TESTING_DOCUMENTATION.md` for full test matrices and trial evidence).

---

1. Quality Assurance Strategy & Test Methodology

The testing architecture for the Campus Equipment Borrowing and Inventory System was developed using a multi-tiered verification methodology:

1. **Unit Testing:** Validating individual model methods, form clean routines, custom template tags, and policy validators in complete isolation.
2. **Service & Transaction Testing:** Verifying multi-table state changes wrapped in `@transaction.atomic`, confirming that partial failures leave zero residual records.
3. **HTTP Client & Integration Testing:** Simulating end-to-end browser requests across both Basic User (`/portal/`) and Admin (`/manage/`) interfaces, including CSRF token validation, session cookies, and pagination controls.
4. **Role-Based Access Control (RBAC) Security Probes:** Explicitly attempting unauthorized actions (e.g., Basic Users invoking admin endpoints, deactivated accounts accessing active sessions, self-promotion attempts via manipulated POST bodies).
5. **Concurrency & Race Condition Trials:** Multi-threaded stress testing of scarce inventory contention (competing approvals for the last remaining unit).
6. **Database Integrity & Migration Consistency:** Applying forward migrations from scratch on empty databases and running foreign key consistency checks (`PRAGMA foreign_key_check`).

---

2. Summary of Automated Test Execution

The integrated test suite was executed across all 6 applications using Django's test runner on October 25, 2026.

```
.\.venv\Scripts\python.exe manage.py test
```

Execution Outcome:

- **Total Tests Discovered:** 516
- **Total Tests Passed:** 516
- **Failures:** 0
- **Errors:** 0
- **Skipped:** 0
- **Execution Duration:** 31.902 seconds
- **Database Status:** Isolated SQLite in-memory/temporary test database destroyed cleanly.

---

3. Comprehensive Breakdown by Application Module

3.1 `apps.accounts` (80 Tests)

Focuses on authentication, password security, session validity, profile management, and admin user administration.

Test Class	Test Count	Scope & Coverage
`EnvironmentConfigurationTests`	2	Verifies development environment safety, settings invariants, and configuration checks.
`RoleConstraintTests`	1	Verifies database-level check constraint `accounts_valid_campus_role`.
`RoleMigrationSafetyTests`	1	Tests backward/forward migration consistency for custom user role definitions.
`TableDateTimeTests`	3	Verifies timezone-aware formatting (`Asia/Manila`) on user activity timestamps.
`StylesheetCacheTests`	3	Verifies cache-busting version query parameters on static assets.
`AccountManagementTests`	16	Admin listing, search, pagination, activation, and deactivation workflows.
`AccountServiceSecurityTests`	14	Verifies deactivated user eviction, protection against privilege escalation, and owner scoping.
`AuthenticationTests`	14	Login, logout, registration form validation, password strength, and role redirection.
`ConcurrentAccountTests`	3	Tests simultaneous account updates and deactivations.
`LandingPageTests`	5	Public landing page navigation, session redirection, and role routing.
`PaginationComponentTests`	4	Verifies 10-record pagination, query string preservation, and page boundaries.
`ProfileTests`	14	Self-service profile updates, password change, and immutability of role/active status.

3.2 `apps.borrowing` (111 Tests)

Focuses on borrowing request submission, multi-item validation, hard limits, admin approval, unit reservation, and cancellations.

Test Class	Test Count	Scope & Coverage
`ApprovalMigrationSafetyTests`	1	Schema consistency for approval reason and timestamp columns.
`RequestComplianceTests`	7	Verifies full compliance with Part 2 (A3) and Part 3 (U3) requirements.

Test Class	Test Count	Scope & Coverage
`DueDateRevisionConcurrencyTests`	3	Tests simultaneous administrative revisions of agreed return dates.
`DueDateRevisionTests`	13	Verifies recorded administrative due date adjustments before physical issue.
`BorrowingHardItemLimitTests`	5	Verifies strict enforcement of the 3-item ceiling and positive quantity validation.
`BorrowingPageTests`	21	Queue filtering (status, search), detail views, and action confirmations.
`BorrowingServiceTests`	30	Atomic request submission, approval unit assignment, rejection, and cancellation.
`ConcurrentBorrowingTests`	3	Simultaneous approval requests and reservation locking.
`DemoBorrowingTests`	3	Repeatable demo seeding and state initialization.
`DynamicEquipmentFormTests`	9	FormSet dynamic row validation and duplicate catalog item prevention.
`EquipmentPreselectionFlowTests`	7	Catalog pre-selection into new request form navigation.
`RequestSummaryPresentationTests`	4	Badge formatting, workflow labels, and empty states.
`TwoStepBorrowingPageTests`	5	Multi-step request interface validation and CSRF protection.

3.3 `apps.dashboard` (76 Tests)

Focuses on live metric derivation, executive charts, CSV reporting, audit logging, and borrowing policy management.

Test Class	Test Count	Scope & Coverage
`DashboardComplianceTests`	5	PDF compliance checks for live cards, monthly trends, and category usage.
`BorrowingItemLimitMigrationTests`	4	Database migrations for singleton policy settings (`SystemSetting`).
`AuditIntegrationTests`	11	End-to-end event logging across all 24 system operations.
`AuditViewTests`	6	Admin-only searchable audit trail view, filters, and pagination.
`DashboardAccessTests`	7	RBAC access controls preventing Basic Users from entering `/manage/` .
`PersonalDashboardTests`	4	Basic User portal statistics (active requests, personal loans).
`ReportsTests`	12	Date-filtered loan reports, overdue calculations, and CSV export streaming.
`SettingsTests`	17	Borrowing limits configuration (`max_items` , `max_quantity` , `max_days`).
`StatisticsAndChartsTests`	10	Dynamic aggregation queries verifying zero static/hardcoded demonstration figures.

3.4 `apps.inventory` (107 Tests)

Focuses on categories, catalog masters, physical unit tracking, asset tags, archiving, and inventory reconciliation.

Test Class	Test Count	Scope & Coverage
`CategoryArchiveTests`	7	Category soft-deletion, archiving behavior, and equipment preservation.

Test Class	Test Count	Scope & Coverage
`InventoryMigrationSafetyTests`	1	Forward/backward migration validation for physical unit metadata.
`PhysicalUnitCategoryFilterTests`	5	Admin multi-attribute filtering (category, catalog model, status).
`PhysicalUnitMetadataTests`	7	Asset tag formatting, physical condition ratings, and acquisition dates.
`ProjectDemoTests`	4	Automated verification of `seed_project_demo` management command.
`InventoryIntegrityAuditTests`	9	Referential integrity protection (`PROTECT`) and duplicate prevention.
`CategoryModalTests`	5	Admin modal dialogs for category creation and modification.
`ConcurrentInventoryTests`	4	Simultaneous asset tag creation and category edits.
`InventoryPageTests`	26	Public catalog browsing, pagination, search, and admin management pages.
`InventoryServiceTests`	29	Stock calculations, serviceable vs retired units, and master archiving.
`PhysicalUnitModalTests`	6	Unit registration modal workflows and validation alerts.
`ServiceableInventoryTests`	4	Verifies 5-state inventory reconciliation against physical stock.

3.5 `apps.loans` (67 Tests)

Focuses on equipment release, printable slips, partial returns, return condition grading, and overdue tracking.

Test Class	Test Count	Scope & Coverage
`CalendarOverdueTests`	5	Boundary tests for calendar date differences in `Asia/Manila` timezone.
`ConcurrentLoanTests`	2	Competing releases and duplicate issue prevention.
`DemoFixtureTests`	3	Demo loan seeding and state verification.
`FoundationTests`	4	Basic model setup, string representations, and foreign key rules.
`LoanPageTests`	22	Loan management queue, release verification, and return inspection views.
`LoanServiceTests`	28	Release execution, unit dispatch, partial returns, and loan completion.
`LoanSummaryPresentationTests`	3	Printable borrowing slip rendering and equipment summary tables.

3.6 `apps.maintenance` (75 Tests)

Focuses on repair intake, damaged return quarantine, diagnostic tracking, unit restoration, and asset retirement.

Test Class	Test Count	Scope & Coverage
`MaintenanceBorrowingGuardTests`	1	Prevents allocated, reserved, or issued units from entering maintenance.
`MaintenanceQuickActionTests`	11	One-click maintenance transitions (Start Repair, Complete, Retire).
`ConcurrentMaintenanceTests`	4	Simultaneous maintenance ticket updates on the same physical unit.
`ExistingMaintenanceMigrationTests`	3	Data migration safety for historical maintenance records.

Test Class	Test Count	Scope & Coverage
`MaintenanceIntegrationTests`	12	End-to-end integration: Damaged return → Quarantine → Repair → Available.
`MaintenanceServiceTests`	23	Ticket creation, diagnostic notes, admin attribution, and state changes.
`MaintenanceSummaryPresentationTests`	2	Maintenance badge styling and ticket history summaries.
`MaintenanceViewTests`	19	Admin maintenance dashboard, filtering, detail views, and resolve dialogs.

---

4. Concurrency Verification: The Last-Unit Allocation Trial

In direct fulfillment of Part 5, Business Rule 3 (BR3) of the IT 401W specifications:

> *"Demonstrate two competing approvals for the last unit: only one succeeds."*

Empirical Trial Evidence:

- **Experiment Setup:** Two distinct pending borrowing requests competing simultaneously for the final single available unit of an equipment model.
- **Trial Repetitions: 30 independent, automated, simultaneous trials**.
- **Results:**
- Exactly **30 out of 30** trials resulted in exactly **one approved request** and **one clean rejection**.
- Winner received full reservation with active allocation.
- Loser raised `django.core.exceptions.ValidationError` ("Selected units are no longer available") with zero residual allocations.
- Zero database lock timeouts, zero deadlocks, and zero orphaned records.

---

5. Security & Boundary Verification Matrices

Test Scenario	Input / Action	Expected Result	Verified Result
Basic User Self-Promotion	Form POST with injected `role=ADMIN`	Role forced to `BASIC_USER`; DB constraint rejects invalid role	**PASS**
Admin Route Direct Access	Unauthenticated or Basic User GET to `/manage/`	302 Redirect to `/login/` or 403 Forbidden	**PASS**
Cross-User Data Leakage	Basic User GET to `/portal/requests/{other_user_id}/`	404 Not Found (Owner-scoped QuerySet)	**PASS**
Excess Items in Request	FormSet submitting 4 distinct catalog items	Form validation error; transaction aborted	**PASS**
Past Due Date Submission	Due date earlier than current server time	Form validation error (`due_at > requested_at`)	**PASS**
Expired Approved Request	Admin attempts to release request past due date	Handover validation blocks issue; requires date revision	**PASS**

Test Scenario	Input / Action	Expected Result	Verified Result
Damaged Unit Return	Return recorded with `condition=DAMAGED`	Unit moves to `MAINTENANCE`; ticket auto-created	**PASS**
Duplicate Return Attempt	Attempting to return already returned unit	View disables returned units; service raises error	**PASS**

Chapter 8 — Conclusion & Future Recommendations

8.1 Project Conclusion

The Campus Equipment Borrowing and Inventory System successfully delivers an institutional-grade, reliable, and secure web application fulfilling all academic learning targets (CO1 to CO6) under the IT 401W syllabus. By implementing strict transactional controls, database-level invariants, dual-role isolation, and comprehensive audit trails, the application completely eliminates the risks of double-booking, untracked damage, and unauthorized access inherent in legacy paper-based operations.

8.2 Summary of Implemented Capabilities

- 100% custom Django views, forms, and templates; zero prohibited admin packages.
- All-or-nothing unit allocation preventing inventory contention.
- Seamless partial return processing and automatic damaged unit maintenance routing.
- Real-time executive dashboards, dynamic usage charts, and CSV report exports.
- Immutable audit logging across 24 distinct operational actions.
- Verified test suite containing 516 passing automated test cases.

8.3 Known System Limitations

- **Local Deployment Configuration:** The current codebase is optimized for local evaluation with SQLite and development settings. Multi-node cloud deployments require migrating to PostgreSQL and configuring static asset CDNs.
- **Manual Barcode Entry:** Physical asset tags are verified via keyboard input; hardware barcode/RFID scanner API integration is not yet implemented.

8.4 Recommendations for Future Enhancement

1. **Automated Institutional Email Notifications:** Integrate Django SMTP backend to dispatch automated email reminders 24 hours prior to loan due dates and immediate notices upon overdue classification.
2. **Hardware Barcode & QR Scanner Integration:** Implement client-side camera-based barcode/QR scanning to accelerate counter checkouts and returns.
3. **Single Sign-On (SSO) Integration:** Connect user authentication to institutional identity providers via LDAP, Google Workspace for Education, or SAML 2.0.

8.5 Ethical, Legal, and Institutional Standards Compliance

- **Data Privacy Act of 2012 (Republic Act No. 10173):** Personal data collection is strictly limited to necessary institutional contact information. Sensitive personal information is not collected. Passwords use salted PBKDF2 hashing. Basic users cannot view other borrowers' identities.
- **Intellectual Property & Code Integrity:** All third-party libraries are declared in `requirements.txt` under permissive open-source licenses (BSD, MIT, Python Software Foundation).
- **Institutional AI Policy:** Permitted AI pair-programming assistance was utilized for documentation synthesis, code auditing, and automated test expansion in strict adherence to BulSU CICT ethical guidelines.

Appendix — Documentation Index & Submission Checklist

IT 401W Final Project Documentation — Submission Index

Course: IT 401W Web Programming and Frameworks
Term: 1st Semester, Academic Year 2026–2027
Institution: Bulacan State University, College of Information and Communications Technology (CICT)
Instructor: Dr. Aaron Paul M. Dela Rosa
Project: Campus Equipment Borrowing and Inventory System (Final Project A)
Architecture: Django Model-Template-View (MTV) Architecture

---

1. Overview & Directory Structure

This directory (`docs/final-submission/`) contains the complete, academic-ready technical documentation package for the **Campus Equipment Borrowing and Inventory System**. All architectural specifications, database schemas, workflow flowcharts, testing records, and analytical chapters directly reflect the verified Django implementation of the system.

```
docs/final-submission/
├── README.md                    # Documentation index, submission guide, and manual checklist
├── PROJECT_DOCUMENTATION.md     # Comprehensive academic manuscript (Chapters 1 to 8)
├── DATABASE_DESIGN.md           # Database schema, table relationships, and data dictionary
├── ERD.md                       # Complete Mermaid Entity Relationship Diagram (ERD)
├── SYSTEM_ARCHITECTURE.md       # Django MTV architecture diagram, request lifecycle, and tech stack
├── SYSTEM_WORKFLOWS.md          # Detailed workflow diagrams, state machines, and narrative explanations
└── TESTING_DOCUMENTATION.md     # Test methodology, test matrices, and 516 verified test results
```

---

2. Document Descriptions

File Name	Scope & Purpose	Target Course Outcomes
`PROJECT_DOCUMENTATION.md`	**Primary academic document** containing Chapters 1 through 8: Project Introduction, System Requirements (A1–A8, U1–U4, BR1–BR6), System Design & Architectural Options Comparison, Database Design, System Workflows, System Implementation, Testing & Verification, and Conclusion with Ethical & Standards Analysis.	CO1, CO2, CO3, CO4, CO5, CO6
`DATABASE_DESIGN.md`	In-depth relational database documentation: ERD summary, table specifications, field definitions, primary/foreign keys, uniqueness constraints, database-level check constraints, composite indexes, and complete Data Dictionary for all 10 Django models across 6 modular applications.	CO4, CO5
`ERD.md`	Standalone, high-resolution Mermaid Entity Relationship Diagram illustrating entities, attributes, primary keys, foreign keys, cardinality notations (1:N, 1:1), and referential integrity constraints.	CO2, CO5
`SYSTEM_ARCHITECTURE.md`	Architectural breakdown of the Django Model-Template-View pattern, component interaction diagram, request-response lifecycle, role-based middleware/decorator security layers, transaction isolation, and architectural trade-off analysis.	CO1, CO2, CO4

File Name	Scope & Purpose	Target Course Outcomes
`SYSTEM_WORKFLOWS.md`	Visual state transitions and Mermaid sequence/activity flowcharts covering User Registration & Login, Equipment Browsing, Borrowing Submission, Admin Approval/Rejection, Allocation Locks, Equipment Release, Partial/Full Returns, Damaged Return Maintenance, and Category Archiving.	CO2, CO3
`TESTING_DOCUMENTATION.md`	Comprehensive testing report covering the test strategy, execution environment, unit tests, integration tests, concurrency race-condition trials (30/30 isolated trials), security/permission tests, and the breakdown of **516 verified automated tests**.	CO3, CO4

---

3. Team Member Contributions & Roles

In accordance with the IT 401W specification, each member was assigned an end-to-end vertical deliverable including database models, service layers, custom Django views, forms, templates, URL routing, automated tests, and documentation:

Member	Feature Branch	Assigned Subsystem & Deliverable	Primary Application Module
Charles	`charles`	User Authentication, Account Security, Profiles, and Admin User Management (A1, U1, SEC1–5)	`apps/accounts`
Raj	`raj`	Equipment Catalog, Category Archiving, Asset Tagging, and Inventory Reconciliation (A2, U2, DB2–3)	`apps/inventory`
Clark	`clark`	Borrowing Request Submission, Multi-Item Validation, Admin Review, and Atomic Unit Allocation (A3, U3, BR1, BR3–4)	`apps/borrowing`
Janelle	`janelle`	Equipment Issuance, Borrowing Slip Generation, Partial/Full Returns, and Overdue Tracking (A4, A5, U4, BR5–6)	`apps/loans`
Francis	`francis`	Maintenance Intake, Damaged Unit Quarantine, Repair Progress, and Unit Retirement (A6, DB7)	`apps/maintenance`
Miah	`miah`	Executive Dashboard, Dynamic Metrics, CSV Reporting, System Settings Policy, and Audit Trail (A7, A8, DB8)	`apps/dashboard`

---

4. Academic Submission Checklist (IT 401W Final Requirements)

Before final submission of the ZIP package (`IT401W_ProjectA_Section_Group.zip`) and the Week 17 defense, the project team must complete the following manual entries:

- [ ] **Title Page & Student Identifiers:** Fill in the group members' Student Numbers, Full Names, Section, and Group Number on the first page of `PROJECT_DOCUMENTATION.md` and the official submission portal.
- [ ] **Safe Demonstration Credentials:** Ensure instructor credentials for 1 Admin and 2 Basic Users are prepared through the designated private submission channel (do not commit passwords to public Git repositories).
- [ ] **Rotational Leadership Evidence:** Document the rotating sprint leadership logs, weekly goal-setting, peer evaluations, and task assignments in accordance with PO-h and PO-j requirements.
- [ ] **Presentation Rehearsal:** Ensure each of the 6 members has an identifiable 2-minute presentation segment for the 12-minute live demonstration, followed by the 3-minute Q&A defense.
- [ ] **Emerging Technology Evaluation:** Review the technology evaluation section in Chapter 3 comparing server-rendered Django against decoupled SPA/API frameworks for institutional deployment.
- [ ] **Clean Archive Packaging:** Package the repository into `IT401W_ProjectA_Section_Group.zip` ensuring `.venv/` , `__pycache__/` , `db.sqlite3` (if containing local/real data), and personal test logs are excluded.